Práctica: Asistente Virtual para una Peluquería con IA

Objetivo de la práctica

En esta práctica, los estudiantes aprenderán a:

- Crear una **interfaz gráfica** sencilla usando la biblioteca **Tkinter** en Python.
- Interaccionar con una API externa (en este caso, la **API de Google Gemini**) para obtener respuestas generadas por un modelo de IA.
- Ocultar claves API utilizando **variables de entorno** para mayor seguridad.
- Gestionar dependencias del proyecto mediante el archivo **requirements.txt**.
- Trabajar con ficheros **JSON**: Aprenderán cómo manejar las respuestas en formato JSON que retorna la API de Gemini para extraer la información relevante y mostrarla al usuario.
- Comprender el concepto de ***entrenar un modelo*** y cómo usar el ***contexto*** de manera eficiente en las consultas con la API.

Descripción del proyecto

Crearás un **asistente virtual** para una peluquería que, usando la API de Google Gemini, responderá preguntas relacionadas con los servicios, precios, horarios y otros detalles de una peluquería. La aplicación tendrá una interfaz gráfica creada con Tkinter.

Requisitos

- API Key de Google Gemini:
  - Debes obtener una API Key a través de la web https://makersuite.google.com/app/apikey.
  - La clave de API se utilizará para interactuar con los modelos de Google Gemini. Investiga cuál es el más adecuado para tu Proyecto.
- Interfaz gráfica con Tkinter:
  - Usa Tkinter para crear una ventana en la que el usuario podrá escribir preguntas sobre los servicios de la peluquería.
  - La respuesta debe ser generada utilizando el modelo de IA de Google.
- Ocultar la API Key:
  - No dejes la clave API visible en el código. Aprende a usar variables de entorno para proteger la clave.
  - Usaremos el archivo **.env** para almacenar la clave de forma segura.
- Gestionar las dependencias del proyecto:

    - Usa un archivo **requirements.txt** para gestionar las dependencias de tu proyecto (por ejemplo, google-generativeai y tkinter).
- Manejo de archivos JSON:
    - La API de Gemini devolverá respuestas en formato JSON. Debes aprender cómo manejar y procesar estos datos JSON para extraer la información relevante y mostrarla en la interfaz gráfica.
- Comprender el concepto de entrenamiento de modelos y el uso del contexto en las llamadas a la API.

Entrenamiento de Modelos y Uso de Contexto

¿Qué significa entrenar un modelo?

Normalmente, cuando trabajamos con inteligencia artificial, ***entrenar un modelo*** significa proporcionarle un conjunto de datos para que aprenda patrones, conocimientos o tareas específicas. Este proceso puede llevar tiempo y recursos computacionales, y se utiliza para enseñar a la IA a realizar tareas como clasificación, traducción o, en este caso, generación de texto.

Sin embargo, ***entrenar un modelo*** desde cero puede ser un proceso largo y costoso. Por eso, muchas veces es más eficiente utilizar un modelo ya preentrenado, como es el caso de la API de Gemini. Este modelo ya ha sido entrenado con grandes cantidades de datos y puede realizar tareas de generación de texto de manera eficiente.

¿Cómo darle contexto a un modelo sin necesidad de entrenarlo?

En lugar de entrenar un modelo desde cero, podemos darle ***contexto*** directamente a través de un **prompt** largo o proporcionándole un **archivo** con la información necesaria. Esto es más rápido y práctico. El modelo usará esa información proporcionada como base para generar respuestas.

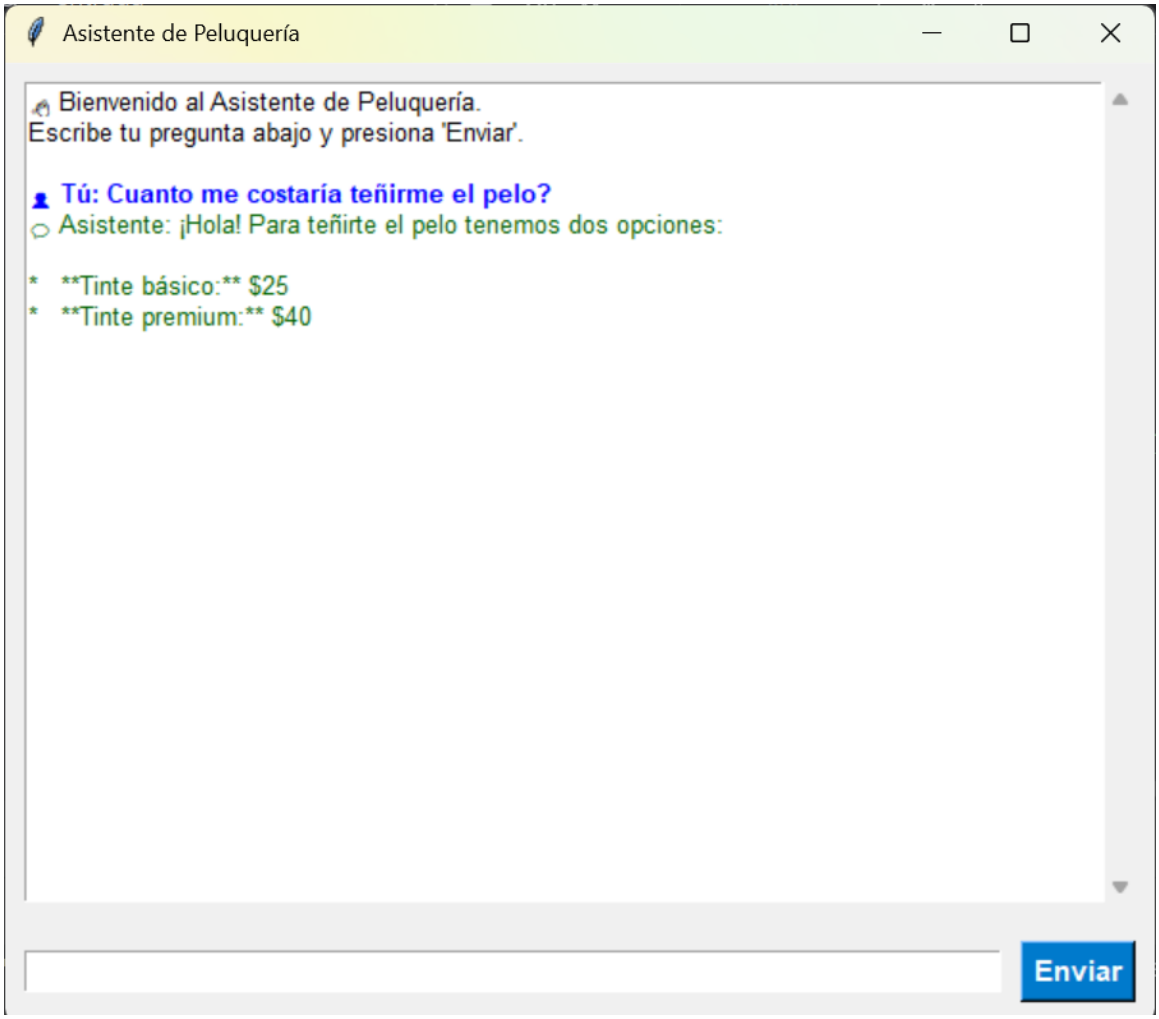
Por ejemplo, en nuestro proyecto, si queremos que el asistente de la peluquería responda preguntas sobre los servicios, precios y horarios, no necesitamos entrenarlo. Podemos simplemente darle ***contexto*** de los servicios y precios escribiéndolo en un **.txt** o **escribiendo** la información directamente **dentro de un prompt**.

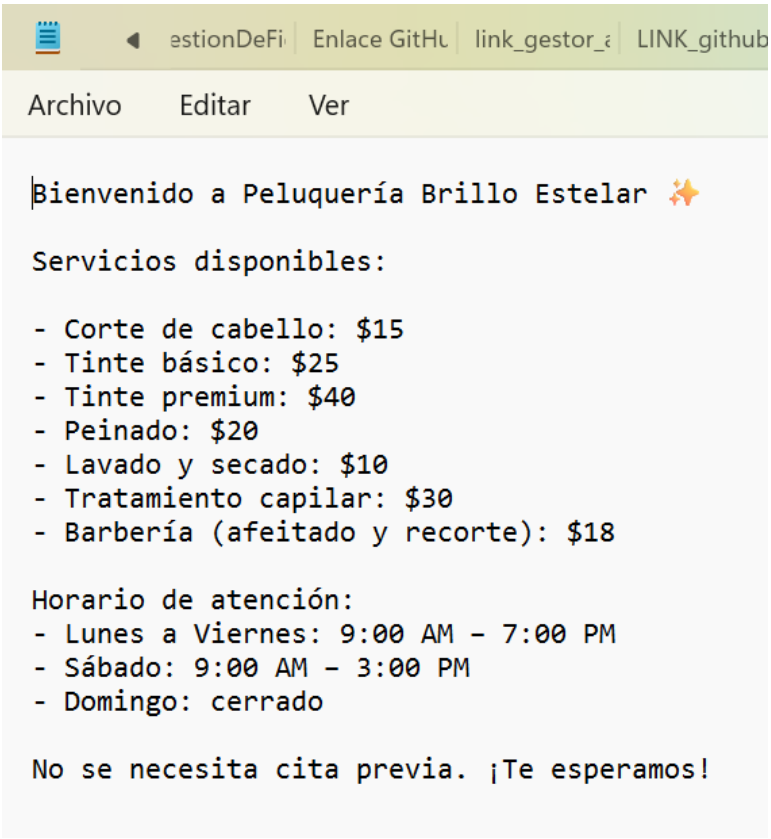
Esto se conoce como ***in-context learning***, y es una técnica poderosa porque permite utilizar modelos generales como Gemini sin tener que entrenar algo específico. El modelo simplemente utiliza el contexto que le proporcionamos para generar respuestas personalizadas.

Pasos a seguir

1. Obtener la API Key de Google Gemini:
   - Regístrate en https://makersuite.google.com/app/apikey.
   - Crea una nueva API Key para tu proyecto de peluquería.
   - Copia la clave generada, pero no la dejes visible en el código.
2. Escribir el código base del asistente virtual:
   - Interfaz gráfica con Tkinter:
     - La aplicación debe tener un cuadro de texto donde el usuario escriba su pregunta (por ejemplo, '¿Cuánto cuesta teñirse el pelo?').
     - Un área de salida para mostrar las respuestas del asistente.
     - Un botón de 'Enviar' para que el usuario pueda hacer la consulta.
3. Conectar con la API de Gemini:
     - Utiliza la API de Gemini para obtener las respuestas a las preguntas.
     - Asegúrate de que las respuestas sean claras y basadas en el contexto de una peluquería.
4. Ocultar la API Key usando Variables de Entorno:
     - Instalar la biblioteca python-dotenv:
       `pip install python-dotenv` o a través de `requirements.txt`
     - Crear un archivo .env en tu proyecto donde guardarás tu clave API:
       ```
       API_KEY=tu_clave_api_aqui
       ```
     - Modificar el código para leer la clave desde el archivo .env:
       ```
       import os
       from dotenv import load_dotenv
       load_dotenv()
       API_KEY = os.getenv('API_KEY')
       ```
5. Gestionar las dependencias del proyecto usando `requirements.txt`:
     - Crear requirements.txt con las dependencias necesarias:
       ```
       google-generativeai
       python-dotenv
       ```
     - Instalar las dependencias con:
       `pip install -r requirements.txt`
6. Ejecutar y probar el asistente:
     - Cuando el programa se ejecute correctamente, la ventana de Tkinter aparecerá con un cuadro de texto y un botón para enviar las preguntas.
     - El asistente debe responder utilizando la API de Gemini.
7. Manejo de respuestas en formato JSON:
     - La API de Gemini devuelve las respuestas en formato JSON. Aprenderás a manejar este tipo de respuesta, extrayendo solo la información relevante para mostrarla en la interfaz gráfica.

Ejemplo


Asistente de Peluquería
Bienvenido al Asistente de Peluquería.
Escribe tu pregunta abajo y presiona 'Enviar'.
Tú: Cuanto me costaría teñirme el pelo?
Asistente: ¡Hola! Para teñirte el pelo tenemos dos opciones:
* **Tinte básico:** $25
* **Tinte premium:** $40
Enviar



Documentación útil

- Documentación de Tkinter:
  - Documentación oficial de Tkinter
  - Tutorial básico de Tkinter en Python
- Documentación oficial de la API de Gemini:
  - Documentación de la API de Gemini
- Manejo de JSON en Python:
  - Documentación oficial de JSON en Python
- IA de tu elección

Entrega

- Enlace github al Proyecto: (asistente_gui.py + requirements.txt + servicios.txt).
- No subas tu archivo .env ni la clave API a repositorios públicos para evitar que otras personas puedan usar tu clave.
- Además, deberás responder algunas preguntas rápidas y básicas en clase sobre la práctica. **En caso de no saber contestar, no se corregirá la práctica. En caso de contestar correctamente se procederá a corregir la práctica atendiendo a la siguiente rúbrica.**

RUBRICA

CRITERIO	PESO	EXCELENTE (10-9)	BUENO (8-7)	SUFICIENTE (6-5)	INSUFICIENTE (<5)
1. COMPRENSIÓN Y DEFENSA DEL CÓDIGO (PRIORITARIO)	35%	El alumno explica con fluidez cómo funciona la comunicación con Gemini. Entiende la diferencia entre "entrenar" y "dar contexto". Sabe explicar línea por línea cómo se cargan las variables de entorno. El código tiene comentarios que justifican la lógica, no solo describen la sintaxis.	El alumno entiende el flujo general (GUI → Python → API → GUI). Comenta el código de forma estándar. Si se le pregunta, sabe dónde se gestiona la clave API, aunque titubea en detalles más técnicos sobre el objeto JSON de respuesta.	El alumno tiene una idea vaga. Sabe ejecutarlo, pero le cuesta explicar por qué se usa os.getenv o cómo la IA "sabe" los precios. Los comentarios son escasos o genéricos.	Suspenso automático. El código funciona, pero el alumno no sabe explicar partes críticas. Indicios de código generado por IA sin revisión (usa funciones que no se han dado en clase y no sabe explicarlas).
2. SEGURIDAD (.ENV) Y DEPENDENCIAS	25%	Seguridad impecable: Usa python-dotenv, la API Key NO está en el código y el archivo .env NO se ha subido al repositorio (o está en .gitignore). El requirements.txt permite instalar todo sin errores.	Usa .env correctamente en local, pero ha tenido algún descuido menor en la gestión del repositorio o el requirements.txt tiene versiones innecesarias. La clave no está hardcodeada.	Funciona, pero la implementación es "sucia". El archivo .env está subido a GitHub (riesgo de seguridad) o faltan librerías en el requirements.txt y hay que instalarlas a mano.	Fallo Crítico de Seguridad: La API Key está escrita directamente en el código Python (API_KEY = "AIzaSy..."). Esto anula este criterio por completo.
3. INTEGRACIÓN IA Y CONTEXTO (PROMPT ENGINEERING)	25%	El asistente responde SOLO como peluquería. El alumno ha construido un prompt robusto inyectando los datos de servicios.txt correctamente. Gestiona la respuesta JSON limpia para mostrar solo el texto necesario, sin caracteres extraños.	Conecta con Gemini y responde. A veces alucina (responde cosas fuera de la peluquería) porque el contexto no está bien delimitado. El manejo del JSON es correcto pero básico.	La IA responde, pero ignora a veces los precios o servicios del archivo de texto. Muestra la respuesta "cruda" o con formato Markdown mal procesado en la ventana de chat.	La conexión con la API falla, no lee el archivo de contexto o la IA responde como un asistente genérico sin saber nada de la peluquería.
4. INTERFAZ GRÁFICA (TKINTER) Y EXPERIENCIA	15%	Interfaz limpia y funcional. Tiene scroll si el texto es largo, el botón funciona bien y limpia el input tras enviar. Gestiona errores (ej: si no hay internet, avisa al usuario en vez de cerrarse).	La ventana cumple: input, botón y output. Funciona correctamente. Diseño básico estándar sin gestión de errores de conexión.	La interfaz es precaria o tiene bugs visuales (ej: el texto se corta, no se puede hacer scroll, la ventana se congela mientras la IA piensa).	La aplicación gráfica no se abre o crashea al intentar enviar el primer mensaje.

Evaluación

La evaluación de esta práctica se basará en los siguientes puntos:

- Funcionamiento correcto del asistente: La aplicación debe responder de manera adecuada según el contexto de los servicios de la peluquería.
- Uso correcto de las variables de entorno: La API Key no debe aparecer en el código. Los estudiantes deben demostrar que saben cómo proteger sus claves API.
- Uso de requirements.txt: El proyecto debe incluir todas las dependencias necesarias para ejecutar el código, y los estudiantes deben ser capaces de instalar las dependencias usando pip install -r requirements.txt.
- Manejo de JSON: Los estudiantes deben ser capaces de leer y manejar respuestas JSON de la API de Gemini para extraer solo la información relevante.
- Demostrar entendimiento del código: Los estudiantes deben demostrar comprensión del código que han escrito. Esto incluye entender cómo interactúan los diferentes componentes (interfaz gráfica, API, manejo de errores, lectura de respuestas JSON, etc.). Se les puede pedir que expliquen el propósito de cada función, cómo se manejan las respuestas de la API, cómo se oculta la clave API, etc.